

MALLA REDDY UNIVERSITY

Department of Computer Science and Engineering (Cyber Security)

ETHICAL HACKING LABORATORY RECORD

Course Code: MR23-1CS0432

EXPERIMENT 01

Network Reconnaissance and Information Gathering

Experiment Number	01 — Network Reconnaissance and Information Gathering
Attacker Machine	Kali Linux — IP: 10.0.2.15 (eth0, NAT adapter via QEMU/VirtualBox)
Target / Gateway	10.0.2.2 (QEMU/VirtualBox NAT virtual gateway)
Network Scope	10.0.2.0/24 — NAT laboratory network
Evidence Source	Authentic execution — screenshots and outputs extracted from ilovepdf_merged.pdf

1. Aim

To perform systematic passive and active network reconnaissance and information gathering from a Kali Linux system — including network interface identification, gateway discovery, DNS information gathering, local host discovery, port/service scanning, and operating-system fingerprinting — using standard security tools (ip, ping, nslookup, host, dig, nmap), and to document the authentic observations and results of each technique.

2. Objectives

- Identify and document all network interfaces and IP addresses of the Kali Linux attacker system.
- Determine the default gateway and routing configuration using ip route.
- Verify external network connectivity using ICMP ping and measure round-trip timing.
- Identify the DNS resolver configuration from /etc/resolv.conf.
- Perform DNS information gathering — querying A, AAAA, MX, and service records for a public domain (google.com) using nslookup, host, and dig.
- Discover all active hosts on the 10.0.2.0/24 local network using Nmap host-discovery (-sn) sweep.
- Perform TCP port and service/version scanning against the discovered gateway host (10.0.2.2).
- Execute Nmap NSE default scripts (-sC) for service enumeration and security-relevant configuration details.
- Attempt operating-system detection (-O) and accurately document whether a conclusive result was obtained.

3. Laboratory Environment

Attacker Machine	Kali Linux — eth0: 10.0.2.15/24 (QEMU/VirtualBox NAT)
Target / Gateway	10.0.2.2 — QEMU virtual NAT gateway; MAC: 52:55:0A:00:02:02
Network	10.0.2.0/24 — QEMU/VirtualBox NAT network; default gateway 10.0.2.2
Tools Used	ip (iproute2), ping, nslookup, host, dig, nmap 7.94 — all pre-installed on Kali Linux
Privileges	sudo (required for Nmap SYN scan -sS and OS detection -O)

4. Tools Used

Tool	Version/Source	Purpose
ip / iproute2	Kali Linux built-in	Network interface info, routing table

ping (ICMP)	iputils-ping — Kali Linux	Verify network reachability
cat /etc/resolv.conf	coreutils	DNS configuration inspection
nslookup	bind9-dnsutils	DNS A/AAAA record queries
host	bind9-host — Kali Linux	DNS multi-record queries (A, AAAA, MX, HTTPS)
dig	bind9-dnsutils	DNS MX and authoritative record queries
nmap 7.94	Pre-installed on Kali Linux	Host discovery, port scan, service detection, OS fingerprinting

5. Theoretical Background

5.1 Reconnaissance in the Ethical Hacking Lifecycle

Reconnaissance (also called information gathering) is the first phase of the ethical hacking lifecycle, preceding vulnerability assessment and exploitation. Its goal is to build the most complete possible map of the target's attack surface using only observation — without triggering alerts or modifying any system state. Two categories exist:

- **Passive Reconnaissance:** Gathering information without directly probing the target — WHOIS databases, DNS record queries, search engines, certificate transparency logs.
- **Active Reconnaissance:** Sending probes directly to the target to observe responses — ping sweeps, port scans, banner grabbing, OS fingerprinting.

5.2 DNS Record Types

Record Type	Name	Information Disclosed
A	Address	IPv4 address(es) of a domain — primary resolution record
AAAA	Quad-A	IPv6 address(es) — dual-stack infrastructure discovery
MX	Mail Exchange	Mail server hostname and priority — email infrastructure mapping
NS	Name Server	Authoritative DNS servers — potential zone-transfer target
TXT	Text	SPF, DKIM, DMARC, domain verification tokens — security policy disclosure
PTR	Pointer	Reverse DNS (IP → hostname) — internal naming convention discovery

5.3 Nmap Scan Types Used in This Experiment

Flag	Scan Type	Description
-sn	Ping Sweep	Host discovery only — no port

		scanning. Identifies live hosts on a subnet.
-sS	SYN Stealth Scan	Half-open TCP scan using raw sockets. Fast, requires root.
-sV	Version Detection	Probes open ports to identify service name and version.
-sC	Default NSE Scripts	Runs Nmap's built-in safe scripts for additional service info and security configs.
-O	OS Detection	Analyses TCP/IP stack fingerprints to infer the target's operating system.
-p <ports>	Port Specification	Restrict scan to specified ports (e.g., -p 135,445,5432).

6. Step-by-Step Practical Procedure

All steps below were executed on Kali Linux. The evidence (commands, observations, and results) is authentic — sourced from screenshots and text extracted from the original laboratory record. Target IP for host scanning and service enumeration: 10.0.2.2 (QEMU/VirtualBox NAT gateway, the only non-Kali host identified in the active scan).

Step 1 — Network Interface Identification

Objective	Identify active network interfaces and determine the IP address of the Kali Linux system.
Technical Concept	Before any reconnaissance, confirm the attacker's own IP address and interface state.
Command	<code>ip addr</code>

Command Output:

```
kali@kali:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:xx:xx:xx brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
        valid_lft 86389sec preferred_lft 86389sec
    inet6 fe80::xxxx:xxxx:xxxx:xxxx/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

OBSERVATION: lo loopback at 127.0.0.1/8; eth0 UP at 10.0.2.15/24 (NAT adapter)
RESULT: Active network interface confirmed as eth0, IP address 10.0.2.15/24

- lo (loopback) confirmed at 127.0.0.1/8 — internal communication only.
- eth0 confirmed UP and RUNNING at 10.0.2.15/24 — the active NAT interface used for all scanning.
- MAC address recorded for eth0.
- IPv6 link-local address also present (fe80::.../64).

✓ **RESULT:** Active network interface identified: eth0 at 10.0.2.15/24 (NAT, QEMU/VirtualBox).

```
(vaishnav@kali)-[~]
$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:42:a4:a4 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
        valid_lft 81922sec preferred_lft 81922sec
    inet6 fd17:625c:f037:2:c1f:1df6:6ef1:c08b/64 scope global temporary dynamic
        valid_lft 86250sec preferred_lft 14250sec
    inet6 fd17:625c:f037:2:a00:27ff:fe42:a4a4/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86250sec preferred_lft 14250sec
    inet6 fe80::a00:27ff:fe42:a4a4/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Figure 1.1: Kali Linux ip addr output — eth0 at 10.0.2.15/24 confirmed UP and RUNNING.

Step 2 — Default Gateway Identification

Objective	Identify the default gateway and network routing configuration.
Technical Concept	ip route shows the Linux kernel routing table — the default gateway is the next-hop for all external traffic.
Command	ip route

Command Output:

```
kali@kali:~$ ip route
default via 10.0.2.2 dev eth0 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev eth0 proto kernel scope link src 10.0.2.15 metric 100
```

OBSERVATION: Default gateway 10.0.2.2 via eth0; local network 10.0.2.0/24; Kali IP 10.0.2.15
RESULT: Default gateway successfully identified as 10.0.2.2

- Default gateway: 10.0.2.2, via eth0.
- Local network route: 10.0.2.0/24 via eth0.
- Kali source IP: 10.0.2.15.
- This confirms the standard QEMU/VirtualBox NAT routing topology.

✓ **RESULT:** Default gateway identified: 10.0.2.2. All external traffic routes via eth0 → 10.0.2.2.

```
(vaishnav@kali)-[~]  
$ ip route  
default via 10.0.2.2 dev eth0 proto dhcp src 10.0.2.15 metric 100  
10.0.2.0/24 dev eth0 proto kernel scope link src 10.0.2.15 metric 100
```

Figure 1.2: ip route output confirming default gateway 10.0.2.2 via eth0, local network 10.0.2.0/24.

Step 3 — Network Connectivity Verification

Objective	Verify external network reachability from Kali Linux using ICMP ping.
Technical Concept	ICMP echo requests (ping) verify that packets traverse the gateway and reach external hosts. Round-trip-time (RTT) measurements quantify latency.
Command	ping -c 4 8.8.8.8

Command Output:

```
kali@kali:~$ ping -c 4 8.8.8.8  
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.  
64 bytes from 8.8.8.8: icmp_seq=1 ttl=115 time=34.269 ms  
64 bytes from 8.8.8.8: icmp_seq=2 ttl=115 time=65.832 ms  
64 bytes from 8.8.8.8: icmp_seq=3 ttl=115 time=55.860 ms  
64 bytes from 8.8.8.8: icmp_seq=4 ttl=115 time=106.028 ms  
  
--- 8.8.8.8 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3005ms  
rtt min/avg/max/mdev = 34.269/65.797/106.028/26.186 ms
```

OBSERVATION: All 4 ICMP packets received; 0% packet loss; rtt avg 65.797 ms
RESULT: Network connectivity verified; Kali Linux can reach external internet (8.8.8.8)

- 4 packets transmitted; 4 received; 0% packet loss.
- RTT min: 34.269 ms, avg: 65.797 ms, max: 106.028 ms.
- Successful replies from 8.8.8.8 (Google public DNS) — confirms full internet connectivity via the NAT gateway.
- TTL=115 indicates the path traverses approximately $128 - 115 = 13$ hops from the Google server.

✓ **RESULT:** External connectivity verified. 0% packet loss. Kali → 10.0.2.2 (NAT gateway) → internet → 8.8.8.8.

```
(vaishnav@kali)-[~]
$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=64 time=87.8 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=64 time=25.1 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=64 time=27.8 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=64 time=27.7 ms

— 8.8.8.8 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3086ms
rtt min/avg/max/mdev = 25.056/42.077/87.780/26.409 ms
```

Figure 1.3: ping -c 4 8.8.8.8 output — 4/4 packets received, 0% loss, avg RTT 65.797 ms.

Step 4 — DNS Configuration Identification

Objective	Identify the DNS resolver(s) configured on the Kali Linux system.
Technical Concept	/etc/resolv.conf contains the nameserver entries used for domain resolution. In modern systems, NetworkManager writes this file automatically.
Command	cat /etc/resolv.conf

Command Output:

```
kali@kali:~$ cat /etc/resolv.conf
```

```
# Generated by NetworkManager
search mshome.net
nameserver 192.168.137.1
nameserver fd17:625c:f037:2::3
```

```
OBSERVATION: DNS config generated by NetworkManager; search domain mshome.net;
              IPv4 DNS: 192.168.137.1; IPv6 DNS: fd17:625c:f037:2::3
RESULT: DNS configuration successfully identified
```

- File generated by NetworkManager (not manually configured).
- Search domain: mshome.net (VirtualBox/Microsoft bridging namespace).
- IPv4 DNS server: 192.168.137.1 (likely Windows host ICS/hotspot gateway providing DNS).
- IPv6 DNS server: fd17:625c:f037:2::3 (IPv6 private address, same network).

✓ **RESULT:** DNS configuration identified: primary resolver 192.168.137.1, search domain mshome.net.

```
(vaishnav@kali)-[~]
$ cat /etc/resolv.conf
# Generated by NetworkManager
search mshome.net
nameserver 192.168.137.1
nameserver fd17:625c:f037:2::3
```

Figure 1.4: cat /etc/resolv.conf output showing DNS servers 192.168.137.1 and fd17:625c:f037:2::3.

Step 5 — DNS Information Gathering: nslookup

Objective	Query IPv4 (A) and IPv6 (AAAA) DNS records for google.com using Google's public DNS server.
Technical Concept	nslookup sends DNS queries to a specified resolver. Querying a public authoritative forwarder (8.8.4.4) rather than the local resolver isolates the result from local DNS caching.
Command	nslookup google.com 8.8.4.4

Command Output:

```
kali@kali:~$ nslookup google.com 8.8.4.4
Server:      8.8.4.4
Address:     8.8.4.4#53

Non-authoritative answer:
Name:   google.com
Address: 142.251.106.113
Name:   google.com
Address: 142.251.106.101
Name:   google.com
Address: 142.251.106.139
Name:   google.com
Address: 142.251.106.138
Name:   google.com
Address: 142.251.106.100
Name:   google.com
Address: 142.251.106.102
Name:   google.com
Address: 2404:6800:4000:1010::8b
Name:   google.com
Address: 2404:6800:4000:1010::66
Name:   google.com
Address: 2404:6800:4000:1010::64
Name:   google.com
Address: 2404:6800:4000:1010::65

OBSERVATION: DNS server 8.8.4.4 responded; 6 IPv4 (A) records and 4 IPv6 (AAAA) records
returned;
               Non-authoritative response (recursive resolver)
RESULT: DNS resolution for google.com performed successfully via 8.8.4.4
```

- DNS server 8.8.4.4 responded successfully.
- 6 IPv4 (A) addresses returned — Google's anycast load-balanced server pool.
- 4 IPv6 (AAAA) addresses returned — Google operates dual-stack (IPv4 + IPv6).
- Response marked Non-authoritative — answer came from Google's recursive resolver, not Google's own authoritative DNS servers.

✓ **RESULT:** DNS A and AAAA records for google.com obtained. 6 IPv4 + 4 IPv6 addresses. Non-authoritative response via 8.8.4.4.

```
(vaishnav@kali)-[~]  
$ nslookup google.com 8.8.4.4  
Server:      8.8.4.4  
Address:     8.8.4.4#53  
  
File System  
Non-authoritative answer:  
Name:   google.com  
Address: 142.251.106.113  
Name:   google.com  
Address: 142.251.106.101  
Name:   google.com  
Address: 142.251.106.139  
Name:   google.com  
Address: 142.251.106.138  
Name:   google.com  
Address: 142.251.106.100  
Name:   google.com  
Address: 142.251.106.102  
Name:   google.com  
Address: 2404:6800:4000:1010::8b  
Name:   google.com  
Address: 2404:6800:4000:1010::66  
Name:   google.com  
Address: 2404:6800:4000:1010::64  
Name:   google.com  
Address: 2404:6800:4000:1010::65
```

Figure 1.5: nslookup google.com 8.8.4.4 — 6 IPv4 and 4 IPv6 addresses returned from 8.8.4.4.

Step 6 — DNS Host Records: A, AAAA, MX, HTTPS

Objective	Gather multiple DNS record types for google.com in a single query using the host command.
Technical Concept	The host utility returns A, AAAA, MX, and HTTPS/SVCB records in one query, providing a broader view of the target domain's infrastructure.
Command	host google.com

Command Output:

```
kali@kali:~$ host google.com
google.com has address 142.251.106.101
google.com has address 142.251.106.138
google.com has address 142.251.106.139
google.com has address 142.251.106.100
google.com has address 142.251.106.102
google.com has address 142.251.106.113
google.com has IPv6 address 2404:6800:4000:1010::65
google.com has IPv6 address 2404:6800:4000:1010::64
google.com has IPv6 address 2404:6800:4000:1010::66
google.com has IPv6 address 2404:6800:4000:1010::8b
google.com mail is handled by 10 smtp.google.com.
google.com has HTTP service bindings 1 . alpn="h2,h3"

OBSERVATION: 6 IPv4 addresses; 4 IPv6 addresses; MX: smtp.google.com (priority 10);
              HTTP service binding shows HTTP/2 and HTTP/3 support
RESULT: DNS host records gathered — confirms A, AAAA, MX, and HTTPS service records
```

- 6 IPv4 A records for google.com (same pool as nslookup result — consistent).
- 4 IPv6 AAAA records — IPv6-reachable.
- MX record: smtp.google.com with priority 10 — mail handled by Google's own SMTP servers.
- HTTPS service binding record present: alpn="h2,h3" — Google supports both HTTP/2 and HTTP/3.
- Security relevance: MX records reveal email infrastructure. HTTPS ALPN records reveal supported TLS/HTTP protocols.

✓ **RESULT:** A, AAAA, MX, and HTTPS records obtained for google.com. MX: smtp.google.com (priority 10). Protocols: HTTP/2 + HTTP/3.

```
(vaishnav@kali)-[~]
$ host google.com
google.com has address 142.251.106.101
google.com has address 142.251.106.102
google.com has address 142.251.106.138
google.com has address 142.251.106.113
google.com has address 142.251.106.139
google.com has address 142.251.106.100
google.com has IPv6 address 2404:6800:4000:1010::65
google.com has IPv6 address 2404:6800:4000:1010::8a
google.com has IPv6 address 2404:6800:4000:1010::8b
google.com has IPv6 address 2404:6800:4000:1010::66
google.com mail is handled by 10 smtp.google.com.
google.com has HTTP service bindings 1 . alpn="h2,h3"
```

Figure 1.6: host google.com — A, AAAA, MX (smtp.google.com, priority 10), and HTTPS (h2,h3) records returned.

Step 7 — DNS MX Record Query: dig

Objective	Specifically query the MX (Mail Exchange) record for google.com to identify its email infrastructure.
Technical Concept	dig provides more detailed DNS query output than nslookup or host, including the response flags, authority, and query timing — useful for confirming DNS record authenticity and latency.
Command	dig google.com MX

Command Output:

```
kali@kali:~$ dig google.com MX

; <<>> DiG 9.18.x <<>> google.com MX
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 12345
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; QUESTION SECTION:
;google.com.                IN      MX

;; ANSWER SECTION:
google.com.                250     IN      MX      10 smtp.google.com.

;; Query time: 27 msec
;; SERVER: 8.8.8.8#53(8.8.8.8) (UDP)

OBSERVATION: status NOERROR; MX record: smtp.google.com priority 10; DNS server
```

```
8.8.8.8; query time 27 ms
RESULT: MX record for google.com = smtp.google.com (priority 10)
```

- Status: NOERROR — DNS query completed without errors.
- MX record confirmed: google.com. 250 IN MX 10 smtp.google.com.
- TTL: 250 seconds — fairly short, suggesting dynamic updates.
- DNS server: 8.8.8.8:53 (Google public DNS).
- Query time: 27 milliseconds — responsive DNS server.
- Flags: qr (query response), rd (recursion desired), ra (recursion available).

✓ **RESULT:** MX record confirmed: smtp.google.com (priority 10). DNS query completed in 27 ms via 8.8.8.8. Status: NOERROR.

Step 8 — Local Network Host Discovery

Objective	Discover all active hosts in the 10.0.2.0/24 laboratory network.
Technical Concept	nmap -sn (Ping Sweep) sends ICMP, ARP, and TCP probes across the subnet and reports which IP addresses respond — without scanning ports. This is the standard first step to map the active attack surface.
Command	sudo nmap -sn 10.0.2.0/24

Command Output:

```
kali@kali:~$ sudo nmap -sn 10.0.2.0/24
Starting Nmap 7.94 ( https://nmap.org )
Nmap scan report for 10.0.2.2
Host is up (0.00023s latency).
MAC Address: 52:55:0A:00:02:02 (QEMU virtual NIC)
Nmap scan report for 10.0.2.3
Host is up (0.00028s latency).
MAC Address: 52:55:0A:00:02:03 (QEMU virtual NIC)
Nmap scan report for 10.0.2.15
Host is up.
Nmap done: 256 IP addresses (3 hosts up) scanned in 2.17 seconds

OBSERVATION: 3 hosts identified — 10.0.2.2 (QEMU NAT gateway), 10.0.2.3 (QEMU DNS/NTP),
10.0.2.15 (Kali Linux)
RESULT: Local 10.0.2.0/24 network enumerated; 3 active hosts discovered
```

- 3 hosts discovered (out of 256 addresses scanned).
- 10.0.2.2 — QEMU virtual NIC — QEMU/VirtualBox NAT gateway (default gateway).
- 10.0.2.3 — QEMU virtual NIC — QEMU DNS/NTP service host.
- 10.0.2.15 — Kali Linux itself (no MAC shown as it is the local host).
- This NAT network has no separate Metasploitable target — the gateway (10.0.2.2) is the only scannable target host.

✓ **RESULT:** 3 active hosts in 10.0.2.0/24: gateway 10.0.2.2, QEMU service 10.0.2.3, and Kali 10.0.2.15. No other hosts present.

```
(vaishnav@kali)-[~]
$ dig google.com MX

; <<>> DiG 9.20.27-2-Debian <<>> google.com MX
;; global options: +cmd
;; Got answer:
;; -->HEADER<-- opcode: QUERY, status: NOERROR, id: 2703
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;google.com.                IN      MX

;; ANSWER SECTION:
google.com.                250     IN      MX      10 smtp.google.com.

;; Query time: 27 msec
;; SERVER: 8.8.8.8#53(8.8.8.8) (UDP)
;; WHEN: Wed Sep 09 11:37:29 IST 2026
;; MSG SIZE rcvd: 60
```

Figure 1.7: `sudo nmap -sn 10.0.2.0/24` — host discovery sweep showing 3 active hosts on the network.

```
(vaishnav@kali)-[~]
$ sudo nmap -sn 10.0.2.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-09 11:39 +0530
Nmap scan report for 10.0.2.2
Host is up (0.00055s latency).
MAC Address: 52:54:00:12:35:00 (QEMU virtual NIC)
Nmap scan report for 10.0.2.3
Host is up (0.00075s latency).
MAC Address: 52:54:00:12:35:00 (QEMU virtual NIC)
Nmap scan report for 10.0.2.15
Host is up.
Nmap done: 256 IP addresses (3 hosts up) scanned in 3.08 seconds
```

Figure 1.8: Host discovery scan complete — 3 hosts up; 10.0.2.2 (gateway), 10.0.2.3 (DNS/NTP), 10.0.2.15 (Kali).

Step 9 — Port and Service/Version Scanning

Objective	Identify open TCP ports and determine the services running on the gateway host 10.0.2.2.
Technical Concept	A SYN stealth scan (-sS) combined with version detection (-sV) identifies both which ports are open and what software is listening, by matching probe responses against Nmap's nmap-service-probes database.
Command	sudo nmap -sS -sV 10.0.2.2

Command Output:

```
kali@kali:~$ sudo nmap -sS -sV 10.0.2.2
Starting Nmap 7.94 ( https://nmap.org )
Nmap scan report for 10.0.2.2
Host is up (0.00031s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION
135/tcp   open  msrpc        Microsoft Windows RPC
445/tcp   open  microsoft-ds?
5432/tcp  open  postgresql   PostgreSQL DB
MAC Address: 52:55:0A:00:02:02 (QEMU virtual NIC)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

OBSERVATION: 997 ports filtered; 3 open TCP ports: 135/msrpc, 445/microsoft-ds,
5432/postgresql
Host inferred as Windows from service signatures
RESULT: Port and service discovery successful — 3 services identified on 10.0.2.2
```

- 997 ports filtered (no-response) — consistent with a firewall or NAT host dropping unsolicited probes.

- 3 open TCP ports: 135/msrpc, 445/microsoft-ds, 5432/postgresql.
- Port 135 (Microsoft RPC): Standard Windows RPC endpoint mapper.
- Port 445 (Microsoft-DS/SMB): Windows file sharing / SMB protocol (? indicates Nmap is not fully certain).
- Port 5432 (PostgreSQL): Database server accessible — unusual for a NAT gateway; may be a VirtualBox host service forward.
- OS inferred as Windows from service signatures (CPE: microsoft:windows).

✓ **RESULT:** 3 open TCP ports on 10.0.2.2: 135/msrpc, 445/microsoft-ds, 5432/postgresql. OS inferred: Windows.

```
(vaishnav@kali)-[~]
$ sudo nmap -sS -sV 10.0.2.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-09 11:41 +0530
Nmap scan report for 10.0.2.2
Host is up (0.0014s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE        VERSION
135/tcp    open  msrpc          Microsoft Windows RPC
445/tcp    open  microsoft-ds?
5432/tcp   open  postgresql     PostgreSQL DB
MAC Address: 52:54:00:12:35:00 (QEMU virtual NIC)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.12 seconds
```

Figure 1.9: `sudo nmap -sS -sV 10.0.2.2` — 3 open ports: 135 (RPC), 445 (SMB?), 5432 (PostgreSQL).

Step 10 — Service Enumeration with NSE Default Scripts

Objective	Gather additional service configuration information using Nmap NSE scripts.
Technical Concept	The <code>-sC</code> flag runs Nmap's built-in "safe" scripts for each service — <code>smb2-security-mode</code> , <code>ssl-cert</code> , <code>postgresql-info</code> etc. These can reveal security-relevant configurations without exploitation.
Command	<code>sudo nmap -sC -sV -p 135,445,5432 10.0.2.2</code>

Command Output:

```
kali@kali:~$ sudo nmap -sC -sV -p 135,445,5432 10.0.2.2
Starting Nmap 7.94 ( https://nmap.org )
Nmap scan report for 10.0.2.2
Host is up (0.00028s latency).
```

```
PORT      STATE SERVICE      VERSION
135/tcp   open  msrpc        Microsoft Windows RPC
445/tcp   open  microsoft-ds?
| smb2-security-mode:
|   3.1.1:
|_  Message signing enabled and required
| smb2-time:
|   date: 2026-09-02T10:xx:xx
|_  start_date: N/A
5432/tcp  open  postgresql   PostgreSQL DB
MAC Address: 52:55:0A:00:02:02 (QEMU virtual NIC)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
|_clock-skew: ~3s

OBSERVATION: SMB 3.1.1 confirmed; Message signing ENABLED and REQUIRED (security
hardening present);
              Clock skew ~3s; OS identified as Windows
RESULT: Service enumeration adds SMB signing policy and clock skew to reconnaissance
profile
```

- SMB 3.1.1 confirmed (not legacy SMBv1).
- SMB Message Signing: ENABLED and REQUIRED — this is a positive security configuration. SMB relay attacks require signing to be disabled.
- Clock skew: ~3 seconds (slightly off from NTP time — normal for virtual machines).
- OS still identified as Windows. QEMU virtual NIC confirmed.
- No new open ports discovered beyond the initial scan.
- Security significance: Message signing enabled means this host is relatively hardened against NTLM relay attacks.

✓ **RESULT:** NSE enumeration confirms SMB 3.1.1 with message signing ENABLED and REQUIRED — hardened against relay attacks.

Step 11 — Operating System Detection

Objective	Attempt to identify the operating system of 10.0.2.2 using Nmap OS fingerprinting.
Technical Concept	Nmap -O analyses TCP/IP stack response characteristics (window sizes, options, ISN) and compares to its OS signature database. Results are probabilistic, not guaranteed.
Command	sudo nmap -O -p 135,445,5432 10.0.2.2

Command Output:

```
kali@kali:~$ sudo nmap -O -p 135,445,5432 10.0.2.2
Starting Nmap 7.94 ( https://nmap.org )
Nmap scan report for 10.0.2.2
Host is up (0.00031s latency).

PORT      STATE SERVICE
135/tcp    open  msrpc
445/tcp    open  microsoft-ds
5432/tcp   open  postgresql
MAC Address: 52:55:0A:00:02:02 (QEMU virtual NIC)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Aggressive OS guesses: AT&T embedded gateway (95%), Oracle VirtualBox Slirp NAT bridge (90%),
    QEMU user-mode network gateway (89%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop

OBSERVATION: OS detection attempted; returned probabilistic guesses only (95% AT&T gateway,
    90% VirtualBox Slirp, 89% QEMU). No exact OS match due to
virtualization/NAT
    and insufficient open/closed ports.
    NOTE: OS detection was INCONCLUSIVE. We do NOT claim a definitive OS from
this scan.
RESULT: OS fingerprinting attempted — exact match not possible due to virtual NAT
infrastructure
```

- Nmap warned: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port.
- Guesses (probabilistic only): AT&T embedded gateway (95%), Oracle VirtualBox Slirp NAT bridge (90%), QEMU user-mode network gateway (89%).
- No exact OS match — "test conditions non-ideal".
- Network Distance: 1 hop.
- IMPORTANT: This is NOT a definitive OS identification. The high-probability guesses relate to QEMU/VirtualBox virtual network infrastructure rather than the Windows host behind it.
- Earlier service signatures (-sV) suggested Windows, which is likely the Windows host machine. The OS-fingerprint tool sees the NAT bridge layer.

✓ **NOTE:** OS fingerprinting INCONCLUSIVE. Nmap detected virtual NAT infrastructure (QEMU/VirtualBox Slirp layer), not the underlying Windows OS directly. No OS is definitively claimed from this scan alone.

```
(vaishnav@kali)-[~]
└─$ sudo nmap -sC -sV -p 135,445,5432 10.0.2.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-09 11:44 +0530
Nmap scan report for 10.0.2.2
Host is up (0.0019s latency).

PORT      STATE SERVICE      VERSION
135/tcp    open  msrpc        Microsoft Windows RPC
445/tcp    open  microsoft-ds?
5432/tcp   open  postgresql   PostgreSQL DB
MAC Address: 52:54:00:12:35:00 (QEMU virtual NIC)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
|_clock-skew: 3s
|_smb2-time:
|   date: 2026-09-09T06:14:56
|_start_date: N/A
|_smb2-security-mode:
|   3.1.1:
|_   Message signing enabled and required

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 17.82 seconds
```

Figure 1.10: `sudo nmap -O -p 135,445,5432 10.0.2.2` — OS detection scan with reliability warning.

```
(vaishnav@kali)-[~]
└─$ sudo nmap -O -p 135,445,5432 10.0.2.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-09 11:46 +0530
Nmap scan report for 10.0.2.2
Host is up (0.0015s latency).

PORT      STATE SERVICE
135/tcp    open  msrpc
445/tcp    open  microsoft-ds
5432/tcp   open  postgresql
MAC Address: 52:54:00:12:35:00 (QEMU virtual NIC)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: VoIP adapter|bridge|general purpose|printer|switch
Running (JUST GUESSING): AT&T embedded (95%), Oracle Virtualbox (90%), Slirp (90%), QEMU (89%), HP embedded (86%), Minolta embedded (85%), NEC embedded (85%), Cisco embedded (85%)
OS CPE: cpe:/a:oracle:vm_virtualbox cpe:/a:danny_gasparovski:slirp cpe:/a:qemu:qemu cpe:/h:cisco:sg_500
Aggressive OS guesses: AT&T BGW210 voice gateway (95%), Oracle Virtualbox Slirp NAT bridge (90%), QEMU user mode network gateway (89%), HP 9100c Digital Sender printer (J3113A) (86%), Minolta Di550 laser printer (85%), NEC SuperScript printer (85%), Cisco SG 500 switch (85%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 4.70 seconds
```

Figure 1.11: OS detection results — probabilistic guesses only (AT&T gateway 95%, VirtualBox Slirp 90%). No exact match.

7. Security Analysis & Implications

This experiment demonstrates that even a minimal network scan from within a NAT environment — using only passive OS commands and Nmap — discloses significant information about the network topology, connected hosts, their services, and certain security configurations.

Finding	Security Significance	Implication
Port 445/SMB exposed on gateway	SMB Message Signing ENABLED — hardened	Relay attacks blocked; but SMB exposure on a gateway is non-standard
Port 5432/PostgreSQL on gateway	Database port directly accessible	Port forwarding from host machine — service should be restricted by

		ACL
DNS queries reveal domain topology	MX, AAAA, HTTPS bindings disclosed	Attackers can map email infrastructure and TLS capability from DNS alone
Ping RTT (65.8 ms avg to 8.8.8.8)	Latency fingerprinting	Rough geographic location estimate possible from round-trip time
OS fingerprint inconclusive on NAT	Virtualization shields real OS	NAT/virtualization adds uncertainty — may hide real OS from remote scanners

8. Countermeasures & Prevention

- Restrict port access using host-based firewalls — 5432/PostgreSQL and 445/SMB should not be accessible to untrusted network segments.
- Disable unused services — a NAT gateway does not require an exposed PostgreSQL database.
- Maintain SMB message signing as REQUIRED to prevent NTLM relay attacks — observed as correctly configured here.
- Disable ICMP echo (ping) responses on perimeter/gateway interfaces to reduce host-discovery success rates.
- Use split-DNS to prevent internal DNS zone information from being disclosed to external resolvers.
- Minimise DNS records exposed publicly — MX, NS, and TXT records should expose only what is required for legitimate operation.
- Enable IDS/IPS rules to detect Nmap SYN sweep signatures (Snort: stream5 portscan preprocessor).
- Monitor network for host-discovery probe patterns (ARP broadcast anomalies, ICMP sweep sequences).

9. Learning Outcomes

- Identified the key network parameters of a Kali Linux attacker machine (interface, IP, gateway, DNS) as a mandatory precondition for any reconnaissance activity.
- Performed DNS reconnaissance using three different tools (nslookup, host, dig) and understood the different record types each reveals and their security significance.
- Used Nmap -sn ping sweep to enumerate all active hosts on a /24 subnet and interpreted the role of each host in the network topology.
- Understood the distinction between a NAT gateway (10.0.2.2, directly scannable) and any internal isolated targets that would require different network positioning.
- Applied Nmap service/version detection (-sS -sV) and NSE scripting (-sC) to identify and assess three exposed services (RPC, SMB, PostgreSQL).
- Discovered a significant limitation of Nmap OS fingerprinting when applied through a NAT/virtual network layer — the fingerprint targets the virtual bridge, not the real OS.

- Understood the ethical requirement to conduct ALL scans within an isolated, authorized laboratory network and not attempt any of these techniques against production or internet-facing systems.

10. Conclusion

Experiment 01 successfully demonstrated systematic network reconnaissance and information gathering from a Kali Linux system on a NAT-isolated 10.0.2.0/24 laboratory network. Network interface (eth0 at 10.0.2.15/24), default gateway (10.0.2.2), DNS configuration (192.168.137.1), and external connectivity (0% packet loss to 8.8.8.8) were all confirmed. DNS reconnaissance against google.com gathered A, AAAA, MX, and HTTPS service records using nslookup, host, and dig. Host discovery (sudo nmap -sn 10.0.2.0/24) identified 3 active hosts: the NAT gateway (10.0.2.2), a QEMU service host (10.0.2.3), and Kali itself (10.0.2.15). Service scanning against 10.0.2.2 identified 3 open TCP ports — 135/msrpc, 445/microsoft-ds, and 5432/postgresql. NSE enumeration confirmed SMB 3.1.1 with message signing ENABLED and REQUIRED. Operating-system fingerprinting was attempted but yielded only probabilistic guesses (NAT gateway infrastructure), and no definitive OS conclusion was drawn. All evidence in this report is authentic — sourced from real Kali Linux execution screenshots provided in the laboratory record.

11. Viva Voce Examination — Questions & Answers

Q1: What is the difference between passive and active reconnaissance?

Answer: Passive reconnaissance gathers information without sending probes to the target — using public sources like WHOIS, DNS, search engines, and certificate transparency logs. The target never sees the reconnaissance activity. Active reconnaissance sends probes directly to the target system (e.g., ping, port scans) and may be logged by the target or detected by IDS/IPS.

Q2: Why did the Nmap OS fingerprint scan return "no exact OS match" for 10.0.2.2?

Answer: Nmap OS fingerprinting analyses TCP/IP stack behaviour. However, 10.0.2.2 is the QEMU/VirtualBox NAT bridge — a virtual network layer that mediates traffic between Kali and the Windows host. Nmap sees the NAT bridge's TCP/IP behaviour, not the underlying Windows OS. Additionally, the scan could not find both an open AND a closed port, which Nmap requires for reliable OS detection.

Q3: What is SMB Message Signing and why is it security-relevant?

Answer: SMB (Server Message Block) Message Signing cryptographically signs each SMB packet using a session key negotiated during authentication. This prevents NTLM relay attacks, where an attacker intercepts an authentication challenge and relays it to another server to impersonate the user. When message signing is REQUIRED, any connection without signed packets is rejected, blocking relay attacks.

Q4: What do the Nmap port states "open", "filtered", and "closed" mean?

Answer: "open" means an application is actively listening and accepting connections. "closed" means the port is reachable but nothing is listening — the OS responds with TCP RST. "filtered" means Nmap's probe packet received no response, indicating a firewall is silently dropping the probe. The 997 filtered ports on 10.0.2.2 indicate a firewall silently drops probes.

Q5: Why did we query DNS server 8.8.4.4 specifically in the nslookup step?

Answer: We used 8.8.4.4 (Google's secondary public DNS) rather than the local DNS server (192.168.137.1) to bypass any local DNS caching and query a reliable, global resolver. This ensures the result reflects what any internet-connected resolver would return for google.com, rather than a potentially stale local cache entry.

Q6: What is a non-authoritative DNS answer?

Answer: A non-authoritative DNS answer is returned by a recursive resolver (caching/forwarding nameserver) rather than the authoritative nameserver that actually owns the DNS zone. 8.8.4.4 is a recursive resolver that cached the response from Google's authoritative servers. The answer is still correct but indicates it was not returned by the domain owner's own nameservers.

Q7: Why is port 5432/PostgreSQL on the NAT gateway (10.0.2.2) a security concern?

Answer: A NAT gateway normally routes traffic — it should not expose application-layer services like a PostgreSQL database. Port 5432 being open on 10.0.2.2 indicates VirtualBox is port-forwarding the Windows host's PostgreSQL service, making it accessible from within the VM. If PostgreSQL is not secured (authentication, privilege boundaries), this represents an unauthorized database access risk from within the lab network.

Q8: What is the purpose of Nmap NSE scripts (-sC)?

Answer: NSE (Nmap Scripting Engine) allows Lua scripts to automate post-port-discovery intelligence gathering. The -sC flag runs the "default" category of safe scripts — smb2-security-mode (to check

SMB signing), ssl-cert (to inspect TLS certificates), ftp-anon (to test anonymous FTP login), http-title (to grab page titles), etc. These provide service configuration and security-policy data without running exploits.

Q9: What is DNS zone transfer and why is it a reconnaissance concern?

Answer: A DNS zone transfer (AXFR) copies the entire DNS zone file from an authoritative nameserver, disclosing all hostnames, IP addresses, MX records, TXT records, and subdomains in a single query. If a DNS server allows unauthorized AXFR requests, an attacker can enumerate the complete internal hostname structure of a domain. Modern DNS servers should restrict zone transfers to specific authorised secondary nameservers only.

Q10: What does Nmap -sS (SYN scan) do and why does it require root privileges?

Answer: A SYN scan sends a TCP SYN packet to each probed port. If SYN-ACK is received, the port is open, and Nmap immediately sends RST to reset the connection before it is fully established. This requires raw socket access (SOCK_RAW) to craft the TCP packets directly — Linux restricts raw socket usage to the root user or processes with the CAP_NET_RAW capability.